

Universidad Autónoma de Baja California

Facultad de Ciencias Químicas e Ingeniería



Reporte Técnico PVVC:

Proyecto Plataforma Web Para Solicitud De Medicamentos Magistrales.



Presentan :

Navarro Quintero Ricardo Antonio Matricula 1290505

Guerra Cervantes Omar Eduardo Matricula 1281859

Tirzo Sotero Ricardo Matricula 1282016

Asesor: Licea Sandoval Guillermo

Índice

Índice.....	2
Introducción.....	2
Desarrollo.....	3
Plan de trabajo.....	3
1.- Cumplimiento de actividades del plan de trabajo.....	5
1.1.- Actividad : Recolección y análisis de requerimientos del sistema.....	5
1.2.- Actividad: Registro, Inicio de sesión y Carga de receta médica por medio de fotografía.....	6
1.3.- Actividad: Programar entrega y seguimiento de pedido.....	9
1.4.- Actividad: Verificación de estados de pago, Visualización del ticket y Procesamiento de pago.....	11
1.5.- Actividad: Gestión de roles, Personalización e Historial.....	15
1.6.- Actividad: Notificaciones y Mensajes.....	19
1.7.- Actividad: Pruebas unitarias y de integración y Pruebas de aceptación.....	22
1.8.- Actividad: Despliegue y capacitación de personal y Documentación técnica de la aplicación.....	23
2.- Evidencias fotográficas.....	23
Conclusión.....	25
Anexo.....	26
Referencias.....	30

Introducción

Este reporte está hecho para documentar el proceso de realización del **Proyecto Plataforma Web Para Solicitud De Medicamentos Magistrales**, como probatorio para el comité de evaluación al que va a ser evaluado este documento contando con probatorios que contienen evidencias y justificaciones del cumplimiento de las actividades que se subieron al sistema SIMA, contando también con una relación entre las actividades realizadas y la PUA de las materias que se van a evaluar en este proyecto. Contando con evidencias gráficas de las visitas realizadas al Hospital Infantil de las Californias, la unidad receptora y con evidencias del cumplimiento de actividades.

El proyecto está basado en una plataforma web para el Hospital Infantil de las Californias donde los pacientes pueden solicitar sus medicamentos en línea a través de la carga de su receta en la plataforma, usando un usuario cliente para los pacientes o padres de los pacientes y un usuario administrador para el encargado en farmacia, con la implementación de un seguimiento para el estado de las órdenes de los clientes manteniendo entre el cliente y el administrador medios de comunicación como un espacio para mensajes entre usuario-administrador y un sistema de notificaciones para ambos lados, teniendo también un espacio para el pago de la receta en línea que fue sometida a un cambio. Todo esto para facilitar al usuario su proceso de ordenar recetas debido a que algunos pacientes tienen que venir de lejos para adquirir sus medicamentos y esto puede verse afectado en caso de

pocos o falta de medicamentos, con los sistemas de manejo de órdenes, comunicación y pagos el paciente solo tendría que presentarse solo una vez a farmacia para adquirir sus medicamentos, evitando muchos inconvenientes para adquirirlos.

Desarrollo

Plan de trabajo.

ID de la actividad	Actividad en SIMA	Inicio	Fin
1	Recolección y análisis de requerimientos del sistema.	3 de febrero de 2025	10 de febrero de 2025
2	1.Registro 2.Inicio de sesión 3.Carga de receta médica por medio de fotografía	11 de febrero de 2025	24 de febrero de 2025
3	1.Programar entrega. 2.Seguimiento de pedido.	25 de febrero de 2025	10 de marzo de 2025
4	1.Verificación estados de pago 2. Visualización de ticket. 3. Procesamiento pago.	11 de marzo de 2025	31 de marzo de 2025
5	1.Gestión de roles 2. Personalización 3. Historial	1 de abril de 2025	12 de abril de 2025

6	1.Notificaciones 2.Mensajes	21 de abril de 2025	5 de mayo de 2025
7	Pruebas unitarias y de integración Pruebas de aceptación	6 de mayo de 2025	30 de mayo de 2025
8	Despliegue y capacitación de personal. Documentación técnica de la aplicación.	20 de mayo de 2025	30 de mayo de 2025

1.- Cumplimiento de actividades del plan de trabajo

1.1.- Actividad : Recolección y análisis de requerimientos del sistema

Inicio: 3 de febrero del 2025. Fin: 10 de febrero del 2025.

Probatorio:

A continuación se muestra una parte de la documentación con los requisitos presentados al encargado de la unidad receptora.

Requerimientos Funcionales

A continuación se describen los requerimientos funcionales del sistema

RF1. El sistema permitirá la gestión completa de usuarios (pacientes, administradores, profesionales de salud).

RF1.1 Registro de usuario (paciente).

RF1.1.1 El usuario (paciente) deberá registrarse ingresando:

- Nombre completo
- Correo Electrónico
- Matrícula

RF1.1.2 Al registrarse, el sistema validará que la matrícula y el correo no estén duplicados.

RF1.1.4 Una vez verificado el correo, se creará un registro en la base de datos con rol "Paciente" y estado "Activo".

RF2. El sistema permitirá el inicio de sesión (login) y control de accesos según rol.

RF2.1 Inicio de sesión.

RF2.1.1 El usuario debe poder autenticarse mediante:

- Matrícula o (correo) y una contraseña.

RF2.1. 2 En caso de credenciales inválidas, mostrar mensaje de error "Usuario o contraseña incorrectos".

RF3. El sistema permitirá a cada usuario (paciente, administrador o profesional) ver y editar su perfil.

RF3.1 Visualizar perfil.

RF3.1.1 El paciente , administrador o profesional podrá consultar:

- a) Nombre completo.
- b) Matrícula.
- c) Correo electrónico.

RF3.2 Editar perfil.

RF3.2.1 El usuario podrá actualizar:

Justificación:

Se sometió una lluvia de ideas y otras técnicas de recolección donde se recolectaron los requisitos necesarios para cumplir con los módulos de actividades que se iban a realizar para el hospital teniendo al final una junta con nuestra encargada en la unidad receptora para la aceptación de los requisitos mostrados.

1.2.- Actividad: Registro, Inicio de sesión y Carga de receta médica por medio de fotografía.

Inicio: 11 de Febrero del 2025. Fin: 24 de Febrero del 2025.

Probatorio:

Registro.

Fragmento en el que los datos capturados al registro se suben a la base de datos para ser almacenados.

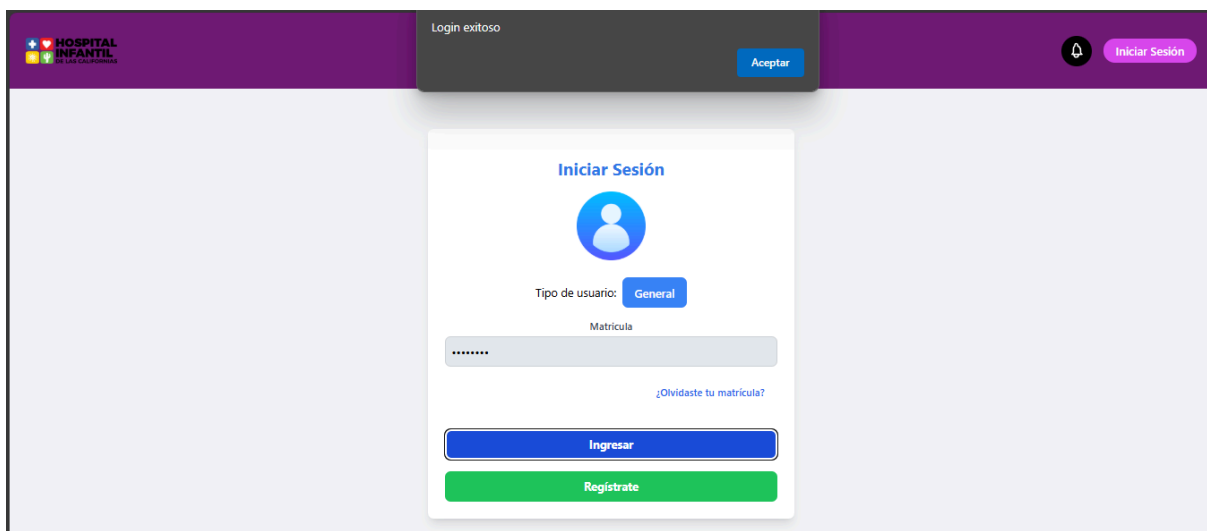
```
24     try {
25         const response = await axios.post("http://localhost:3000/user/register", {
26             name_: state.nombreUsuario,
27             email: state.correo,
28             password_: "default",
29             matricula: state.userMatricula,
30             role: "general",
31         });
32
33         if (response.data.success) {
34             alert("Usuario creado exitosamente.");
35             navigate("/login_g");
36         } else {
37             alert("Error al crear usuario. Por favor, intente de nuevo.");
38         }
39     } catch (error) {
40         alert("Creación de usuario fallida.");
41         console.error("Error durante creación de usuario:", error);
42     }
43 }
```

The screenshot displays a web application interface. At the top, a purple header bar contains the logo for 'HOSPITAL INFANTIL DE LAS CALIFORNIAS' on the left, a dark grey notification box in the center stating 'Usuario creado exitosamente.' with an 'Aceptar' button, and a bell icon and 'Iniciar Sesión' button on the right. Below the header, a white registration form titled 'Registro de Usuario' is centered. The form includes the following fields: 'Nombre completo*' with the value 'Alejandro', 'Correo electrónico (opcional)' with 'Alejandro@gmail.com', 'Matricula*' with '11228899', and 'Confirmar Matricula*' with '11228899'. A blue 'Registrar Usuario' button is at the bottom of the form.

Inicio de sesión.

Muestra del código donde se verifica la matricula del usuario a través de la base de datos.

```
33     const loginGeneral = async () => {
34         try {
35             const response = await axios.post('http://localhost:3000/user/login', {
36                 password: matricula,
37                 rol: "general"
38             });
39             if (response.status === 200) {
40                 const currentUser = response.data.user;
41                 sessionStorage.setItem("usuarioActual", JSON.stringify(currentUser));
42                 alert("Login exitoso");
43             }
44         } catch (error) {
45             alert("Matricula incorrecta");
46             console.error(error);
47         }
48         navigate("/", { replace: true });
49     };
50
51     const handleTypeOfUser = () => {
52         if (activeButton === "general") {
53             setActiveButton("admin");
```



Carga de receta médica por fotografía.

Carga de fotografía.

```
const handleFileChange = e => {
    const selectedFile = e.target.files[0];
    if (!selectedFile) return;

    if (!['image/jpeg', 'image/png'].includes(selectedFile.type)) {
        setErrorMessage("Formato no soportado. Sólo JPEG o PNG.");
        setFile(null);
        return;
    }

    if (selectedFile.size > 10 * 1024 * 1024) {
        setErrorMessage("Archivo demasiado grande. Máx 10MB.");
        setFile(null);
        return;
    }

    setErrorMessage("");
    setFile(selectedFile);
    setImageUrl(URL.createObjectURL(selectedFile));
};
```

Almacenamiento del cuestionario y datos de la receta médica en la base de datos.

```
7 const UploadComponent = () => {
92   const handleUpload = async () => {
93     if (!file) return;
94     // validación de sabores
95     if (flavors.some(f => !f)) {
96       alert("Seleccione un sabor para cada medicamento.");
97       return;
98     }
99
100    const formData = new FormData();
101    formData.append('image', file);
102    formData.append('user_id', currentUser.user_id);
103    formData.append('nombre_completo', nombreCompleto);
104    formData.append('peso', peso);
105    formData.append('fecha_de_nacimiento', fechaDeNacimiento);
106    formData.append('padecimiento', padecimiento);
107    formData.append('i_lactosa', intoleranciaLactosa);
108    formData.append('flavor', JSON.stringify(flavors));
109
110    try {
111      setIsUploading(true);
112      const response = await axios.post(
113        'http://localhost:3000/prescriptions',
114        formData,
115        {
116          headers: { 'Content-Type': 'multipart/form-data' },
117          onUploadProgress: e => {
118            setProgress(Math.round((e.loaded * 100) / e.total));
119          }
120        }
121      );
122
123      if (response.status === 201) {
124        alert(
125          `Receta creada (ID: ${response.data.prescription.prescription_id}).\n` +
126          `El administrador añadirá los nombres de medicamento luego.`
127        );
128        setSuccess(true);
129        setImageUri('http://localhost:3000/${response.data.prescription.image_url}');
```

The screenshot displays a web application interface. At the top, a modal dialog is open with the text: "Receta creada (ID: 10). El administrador añadirá los nombres de medicamento luego." and an "Aceptar" button. Below the modal, the main form is visible. It includes a header "Paciente Omar Guerra" and a section titled "Medicamentos". Under "Medicamentos", there is a text input field containing "1-formula1 500 mg sabor vainilla durante 5 días.". Below this, there is a button labeled "Eliminar Imagen". Further down, there is a section for "Cantidad de medicamentos" with a value of "2". Below that, there are two dropdown menus for "Sabor para medicamento 1" (selected "Mango") and "Sabor para medicamento 2" (selected "Chicle Rosa"). At the bottom, there is a blue button labeled "Procesando..." with a loading spinner.

Justificación:

La siguiente actividad cumple con los requisitos del almacenamiento y despliegue de los datos del usuario al integrar con el Registro una función para almacenar los datos del usuario general en la base de datos y mediante el inicio de sesión se consulta la matrícula almacenada para la verificación de que el usuario que quiere acceder a su cuenta si está ingresando sus datos al sistema o no, mientras que la carga de la receta envia datos vitales para farmacia y mediante otras tecnologías se registra una ruta y se almacena la foto para recuperarla y mostrarla más adelante.

1.3.- Actividad: Programar entrega y seguimiento de pedido.

Inicio: 25 de Febrero del 2025. Fin: 10 de Marzo del 2025.

Probatorio:

Programar orden.

Obtener de la base los datos del paciente en la orden creada

```
const getOrders = async () => {
  try {
    const response = await axios.get("http://localhost:3000/order/", {
      params: { page }
    });
    console.log("Órdenes recibidas:", response.data.orders);
    const ordersWithNames = await Promise.all(response.data.orders.map(async (order) => {
      const patientDetails = await getPatientName(order.user_id);
      return { ...order, patient_name: patientDetails ? patientDetails.user.name : "Desconocido" });
    }));
    setOrders(ordersWithNames);
    setTotalPages(response.data.totalPages);
  } catch (error) {
    alert("No se pudo obtener la informacion de las ordenes");
    console.log("Error al obtener ordenes", error);
  }
};
```

Modificar los datos en la orden para modificar los estados y fecha de entrega

```
const handleSaveChanges = async (order_id, order) => {
  if (!updatedDelivery || isNaN(new Date(updatedDelivery).getTime())) {
    alert("Por favor ingrese una fecha de entrega válida");
    return;
  }

  const updatedOrder = {
    ...order,
    state: updatedState,
    delivery_schedule: updatedDelivery ? new Date(updatedDelivery).toISOString() : null,
    estado_pago: updatedPayment
  };

  try {
    const response = await axios.put(`http://localhost:3000/order/update/${order_id}`, updatedOrder);
    if (response.data.success) {
      alert("Orden actualizada");
      setOrders(prevOrders => prevOrders.map(o =>
        o.order_id === order_id ? { ...o, state: updatedState, delivery_schedule: updatedOrder.delivery_schedule, is_paid: updatedPayment }
        : o
      ));
      setOrderMsg(false);
    }
  } catch (error) {
    alert("Error al actualizar pedido");
    console.log("Error : ", error);
  }
};
```

HOSPITAL INFANTIL

Hola soyadmi

Lista de Ordenes en el Sistema

Detalles de la Orden:

ID: 5

Paciente: Alejandro

Estado: Preparando

Sin pagar

Fecha de pedido: 2011-07-25T17:31:37.000Z

Fecha de entrega: dd/mm/aaaa

Guardar Cambios

Aceptar

Fecha de pedido: 2011-07-20T20:35:22.000Z

Fecha de entrega: 2025-05-12T00:00:00.000Z

Actualizar Orden

ID: 3

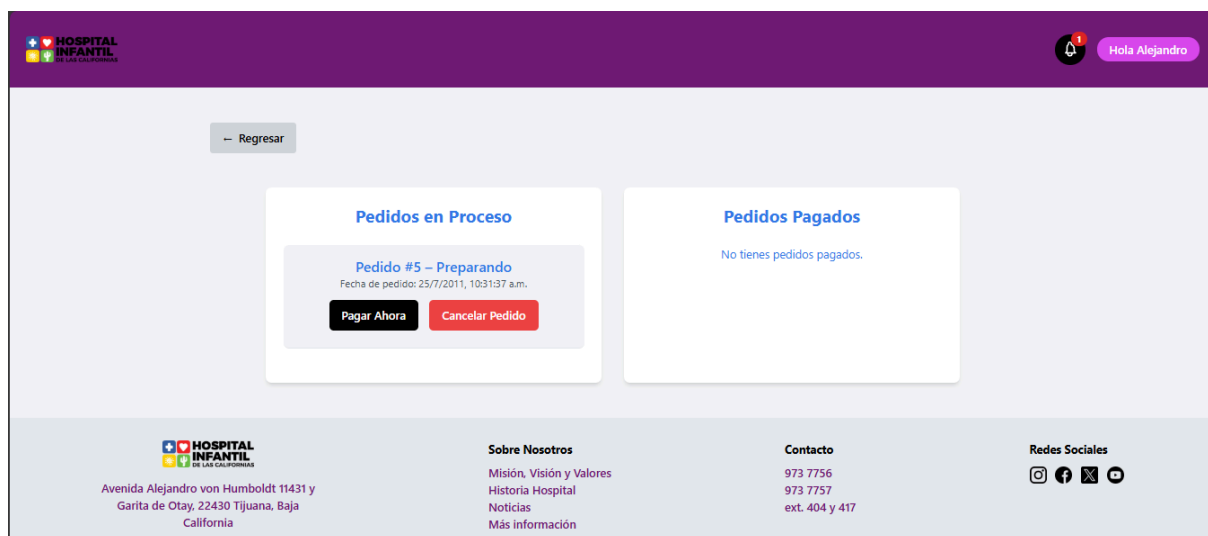
Paciente: JUAN

Seguimiento de pedido

Ordena las órdenes del usuario registradas en la base de datos.

```
const fetchUserOrders = async () => {
  try {
    const usuario = JSON.parse(sessionStorage.getItem('usuarioActual'));
    if (!usuario) throw new Error('Usuario no autenticado');
    const user_id = usuario.user_id;

    const { data } = await axios.get(
      'http://localhost:3000/order', // singular
      {
        params: {
          page: 1,
          limit: 100,
          user_id // para que el backend filtre
        }
      }
    );
  }
};
```



Justificación:

La programación de la orden permite al administrador a manejar los detalles de la orden para indicar el estado en el que se encuentra y los detalles de si está pagada o no, así como también programar una fecha de entrega para los clientes, esto por medio del registro de la prescripción se genera la orden del paciente y en el código se recaba los datos más importantes para realizar la entrega del pedido editando los estados y fechas y el seguimiento de la orden nos mostrará las órdenes que ha pedido el usuario que está usando la plataforma.

1.4.- Actividad: Verificación de estados de pago, Visualización del ticket y Procesamiento de pago.

Inicio: 11 de Marzo del 2025. Fin: 31 de Marzo del 2025.

Probatorio:

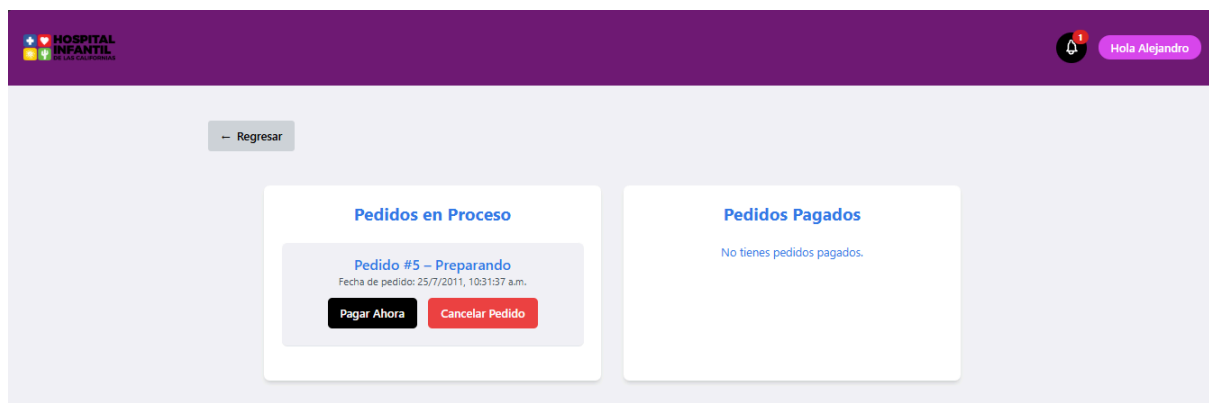
Verificación de estados de pago.

Mostrar y filtrar las órdenes de pago del usuario por pedidos en proceso(sin pagar) y órdenes pagadas.

```
const fetchUserOrders = async () => {
  try {
    const usuario = JSON.parse(sessionStorage.getItem('usuarioActual'));
    if (!usuario) throw new Error('Usuario no autenticado');
    const user_id = usuario.user_id;

    const { data } = await axios.get(
      'http://localhost:3000/order', // singular
      {
        params: {
          page: 1,
          limit: 100,
          user_id // para que el backend filtre
        }
      }
    );

    const inProcess = data.orders.filter(o => o.estado_pago === 'Sin Pagar');
    const paid = data.orders.filter(o => o.estado_pago === 'Pagada');
    setOrders({ inProcess, paid });
  } catch (error) {
    console.error('Error al cargar los pedidos:', error);
  } finally {
    setLoading(false);
  }
};
```



Visualización del ticket.

Obtener los datos para el ticket.

```
const CardPaymentForm = () => {
  const { orderId } = useParams();
  const navigate = useNavigate();
  const [orderData, setOrderData] = useState(null);
  const [paymentMethod, setPaymentMethod] = useState("ventanilla");

  useEffect(() => {
    const fetchOrder = async () => {
      try {
        const { data } = await axios.get(`http://localhost:3000/order/${orderId}`);
        if (data.success) {
          console.log("Datos completos de la orden recibidos:", data);

          const userDetails = await fetchUserDetails(data.order.user_id);
          setOrderData({
            ...data.order,
            username: userDetails.username,
            matricula: userDetails.matricula
          });
        } else {
          console.error("Orden no encontrada");
        }
      } catch (err) {
        console.error("Error al obtener la orden:", err);
      }
    };
  });
};
```

Generar el ticket en la plataforma.

```
Resumen del Pedido
</h1>
<div className="flex justify-center space-x-8 mb-6">
  <p className="text-blue-600 font-semibold">
    Pedido: #{orderData.order_id}
  </p>
  <p className="text-blue-600 font-semibold">
    Fecha: {new Date(orderData.order_date).toLocaleString()}
  </p>
</div>
<ul className="space-y-3">
  {orderData.medicamentos.map(m => (
    <li
      key={m.medicamento_id}
      className="flex justify-between items-center bg-gray-50 rounded p-3"
    >
      <span className="font-medium">
        {m.nombre} {m.flavor}
      </span>
      <span className="font-semibold">
        ${parseFloat(m.precio).toFixed(2)}
      </span>
    </li>
  ))}
</ul>
<div className="mt-6 text-right">
  <span className="text-lg font-bold text-blue-600">
    Total: ${total} MXN
  </span>
</div>
</div>
```





Hola Alejandro

← Regresar

Resumen del Pedido

Pedido: #5

Fecha: 25/7/2011, 10:31:37 a.m.

Test101 (Mango)	\$345.50
Test102 (Chicle Rosa)	\$120.60
Total: \$466.10 MXN	

Seleccionar Método de Pago

Pago en Ventanilla

Transferencia Bancaria

Generar Recibo (Ventanilla)

Procesamiento de pago.

```
try {
  const response = await axios.post(
    "http://localhost:3000/pdf/generate-pdf",
    {
      userName: username,
      userMatricula: user_id,
      pago: `${total}`,
      userOrderID: order_id,
      medicamentos,
      tipoPago
    },
    { responseType: "arraybuffer" }
  );
  console.log("Username:", orderData.username);
  console.log("matricula:", orderData.matricula);

  const blob = new Blob([response.data], { type: "application/pdf" });
  const url = window.URL.createObjectURL(blob);
  const a = document.createElement("a");
  a.href = url;
  a.download = tipoPago === "ventanilla"
    ? `recibo_ventanilla_${user_id}.pdf`
    : `recibo_transferencia_${user_id}.pdf`;
  document.body.appendChild(a);
  a.click();
  a.remove();
} catch (error) {
  console.error("Error al generar el PDF:", error);
  alert("Hubo un problema al generar el recibo");
}
};
```

Escribir en el PDF

```
router.post('/generate-pdf', async (req, res) => {

  // Agregar datos al PDF
  firstPage.drawText(`${userName}`, { x: 118, y: 623, size: 11 });
  firstPage.drawText(`${userMatricula}`, { x: 125, y: 595, size: 11 });
  firstPage.drawText(`${userOrderID}`, { x: 150, y: 565, size: 11 });
  firstPage.drawText(`${pago}`, { x: 145, y: 128, size: 11 });
  let medicamentosTexto = "No hay medicamentos";
  if (Array.isArray(medicamentos)) {
    medicamentosTexto = medicamentos.map(med => `${med.nombre} (${med.flavor})`).join('\n');
  }
  firstPage.drawText(medicamentosTexto, { x: 72, y: 500, size: 11 });
  //firstPage.drawText(`${medicamentos.join(', ')}`, { x: 72, y: 500, size: 11 }); //medicamentos.join(', ')

  // Guardar el nuevo PDF con nombres distintos según el tipo de pago
  const outputFile = tipoPago === 'ventanilla' ? `recibo_ventanilla_${userMatricula}.pdf` : `recibo_transferencia_${userMatricula}.pdf`;
  const outputPath = path.join(__dirname, '..', 'uploads', outputFile);
  fs.writeFileSync(outputPath, await pdfDoc.save());

  res.setHeader('Content-Type', 'application/pdf');
  res.setHeader('Content-Disposition', `attachment; filename=${outputFile}`);
  res.send(Buffer.from(await pdfDoc.save()));

} catch (error) {
  console.error("Error al generar el PDF:", error);
  res.status(500).json({ success: false, message: `Error al generar el PDF: ${error.message}` });
}
});
```



Datos:

Nombre: Alejandro

Matricula: 10

ID de la Orden: 5

Medicamentos:

Test101 (Mango)

Test102 (Chicle Rosa)

Total a pagar: \$466.10

Nota: El pago se realizará en ventanilla de hospital y se deberá corroborar con los encargados en ventanilla que el pago sea actualizado en la preparación de su orden.

Justificación:

Como se observa la verificación del estado de pagos divide en 2 áreas, el estado del pago de la orden, si en la base de datos esta se encuentra como 'sin pagar' esta se mostrará del lado izquierdo caso contrario aparecerá en el derecho como pagada, el usuario al pagar se le generará un ticket obteniendo los datos de la base y los procesa para generar un ticket y el pago final por todos los medicamentos en su pedido, por la parte del procesamiento de pagos se muestra un pdf para el pago en ventanilla debido a que hubo un problema con la API de pagos por parte del hospital, adjuntamos justificación para el cambio por pago en ventanilla, pero se espera en un futuro se pueda realizar su implementación.



farmacia@hospitalinfantil.org
para mí ▾

mar, 27 may, 15:06 (hace 7 días)



Traducir al español



Bonita tarde

Reenvió cuestionario que durante el semestre se les solicito agregar a la plataforma.

Sobre el tema de generar pago queda pendiente verificar este contenido con las áreas administrativas del hospital.

Gracias

Quedo pendiente a sus comentarios



1.5.- Actividad: Gestión de roles, Personalización e Historial.

Inicio: 1 de Abril del 2025. Fin: 12 de Abril del 2025.

Probatorio:

Gestión de roles.

Gestión para el rol general(pacientes/clientes).

```
{(!currentUser || currentUser.role === "general") ? (

to="/cargar-receta"  
onClick={handleOrderClick}  
className="bg-white shadow-lg rounded-xl p-4 sm:p-6 md:p-8 lg:p-10 2xl:p-12 w-full flex flex-col items-center justify-center space-y-3 sm:space-y-4 transition-transform transform hover:scale-105"



text-blue-600 w-10 h-10 sm:w-12 sm:h-12 md:w-16 md:h-16 lg:w-20 lg:h-20 2xl:w-24 2xl:h-24



text-base sm:text-lg md:text-xl lg:text-2xl 2xl:text-3xl font-semibold



Subir Receta


```

En caso de que el rol sea Administrador.

```

    : (
    <
    <Link
      to="/prescriptions"
      className="
        bg-white shadow-lg rounded-xl
        p-4 sm:p-6 md:p-8 lg:p-10 2xl:p-12
        w-full
        flex flex-col items-center justify-center
        space-y-3 sm:space-y-4
        transition-transform transform hover:scale-105
      "
    >
    <Clipboard className="
      text-purple-600
      w-10 h-10 sm:w-12 sm:h-12
      md:w-16 md:h-16
      lg:w-20 lg:h-20
      2xl:w-24 2xl:h-24
    " />
    <span className="
      text-base sm:text-lg md:text-xl lg:text-2xl 2xl:text-3xl
      font-semibold
    ">
      Recetas
    </span>
  </Link>

  <Link
    to="/ver_ordenes"

```

Gestión de usuario general



Hola Alejandro

Hospital Infantil de las Californias



Subir Receta



Seguimiento de Pedido



Historial de Pagos



Mensajes



Avenida Alejandro von Humboldt 11431 y Garita de Otay, 22430 Tijuana, Baja California

Sobre Nosotros

Misión, Visión y Valores
Historia Hospital
Noticias
Más Información

Contacto

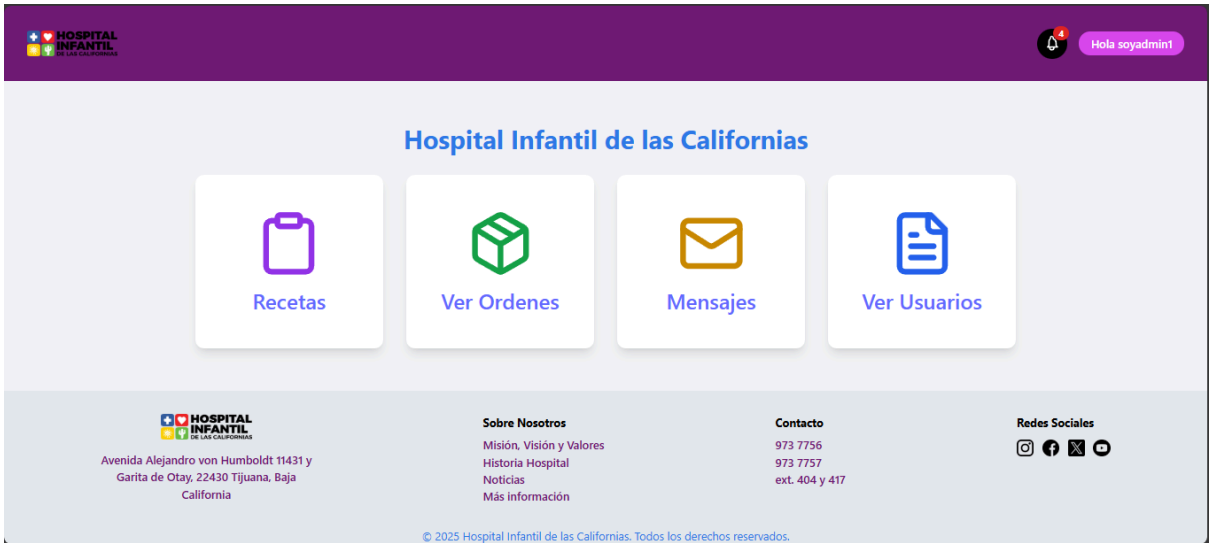
973 7756
973 7757
ext. 404 y 417

Redes Sociales

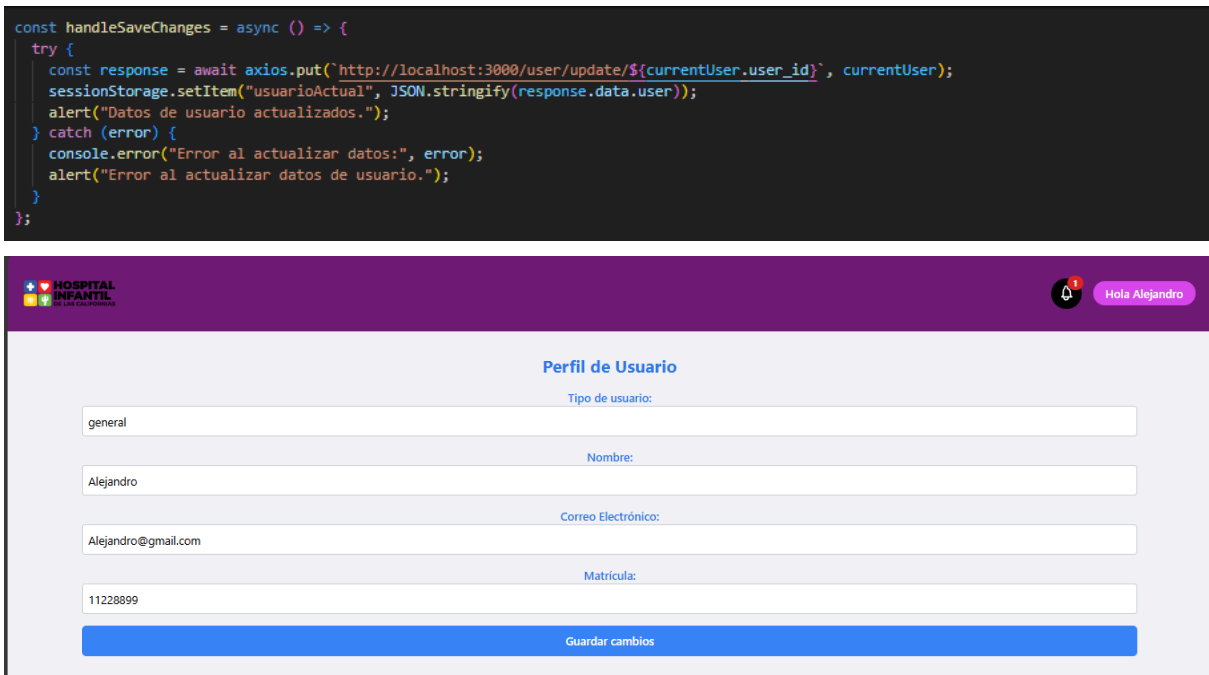


© 2025 Hospital Infantil de las Californias. Todos los derechos reservados.

Gestión de Administrador.



Personalización



Historial

```
const getHistorialPagos = async () => {
  try {
    const response = await axios.get("http://localhost:3000/historial_pagos/", { //orders
      params: { page }
    });
    setHistorialPagos(response.data.historial);
    setTotalPages(response.data.totalPages);
    console.log("Datos Recibidos");
    //alert("Usuario actual:", currentUser);
  } catch (error) {
    alert("No se pudo obtener la información de las órdenes");
    console.log("Error al obtener órdenes");
  }
};

const filterOrders = () => {
  const filtered = HistorialPagos.filter(order => order.user_name === currentUser.name_);
  console.log("Usuario actual:", currentUser);
  setUserOrders(filtered);
};
```

ORDEN ID	IMPORTE	FECHA	CLIENTE	ESTADO
5	\$--	2011-07-25T17:31:37.000Z	Alejandro	Preparando

Justificación:

Para la gestión de roles se muestra una de sus funciones en el apartado de la página principal en la plataforma, donde se muestra como muestra la página dependiendo del usuario que ingresa, a su vez se maneja la personalización de los datos de los usuarios para que estos puedan modificar su información y mantener su cuenta segura. Mientras que el historial mantendrá evidencia de las distintas compras del usuario en la plataforma y el estado en la que estas se encuentran en la base de datos.

1.6.- Actividad: Notificaciones y Mensajes.

Inicio: 21 de Abril del 2025. Fin: 5 de Mayo del 2025.

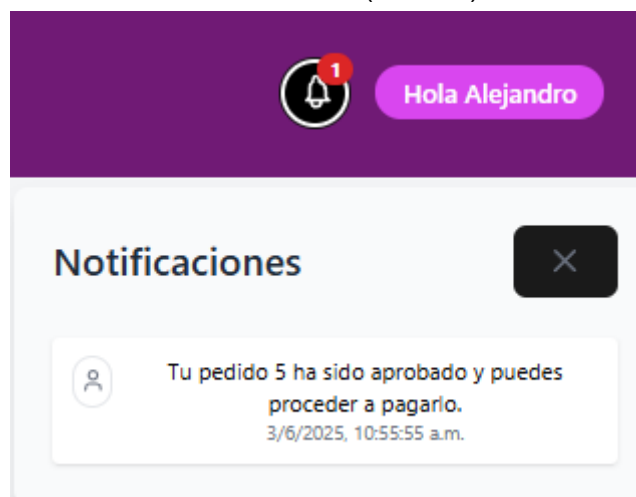
Probatorio:

Manejo para mostrar y leer notificaciones.

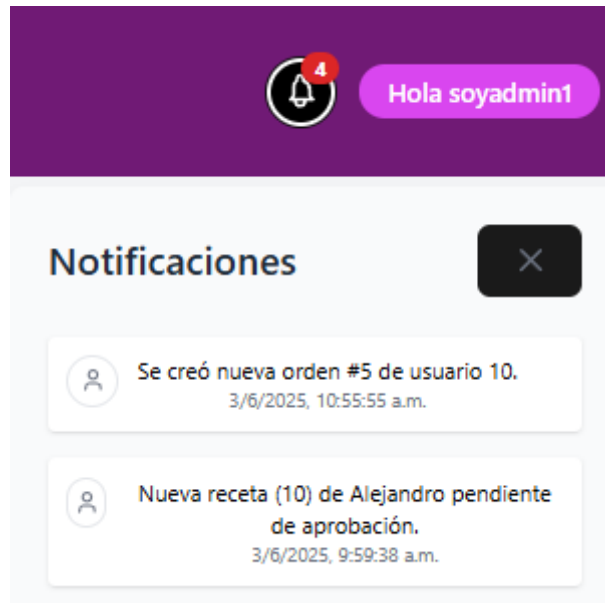
```
useEffect(() => {
  const fetchNotifications = async () => {
    try {
      const { data } = await axios.get(
        `http://localhost:3000/notifications/${userId}`
      );
      setNotifications(data);
    } catch (err) {
      console.error('Error al cargar notificaciones:', err);
    }
  };
  fetchNotifications();
}, [userId, refreshKey]);

const markAsRead = async (id) => {
  try {
    await axios.put(
      `http://localhost:3000/notifications/read/${id}`
    );
    setNotifications((prev) =>
      prev.map((n) =>
        n.notification_id === id ? { ...n, is_read: true } : n
      )
    );
  } catch (err) {
    console.error('Error al marcar como leída:', err);
  }
};
```

Notificaciones (usuario).



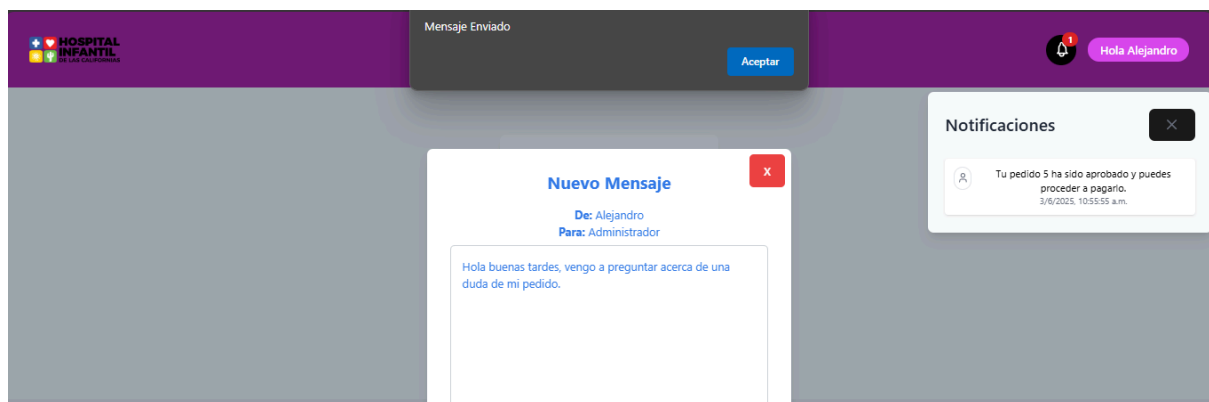
Notificaciones (administrador).



Mensajes.

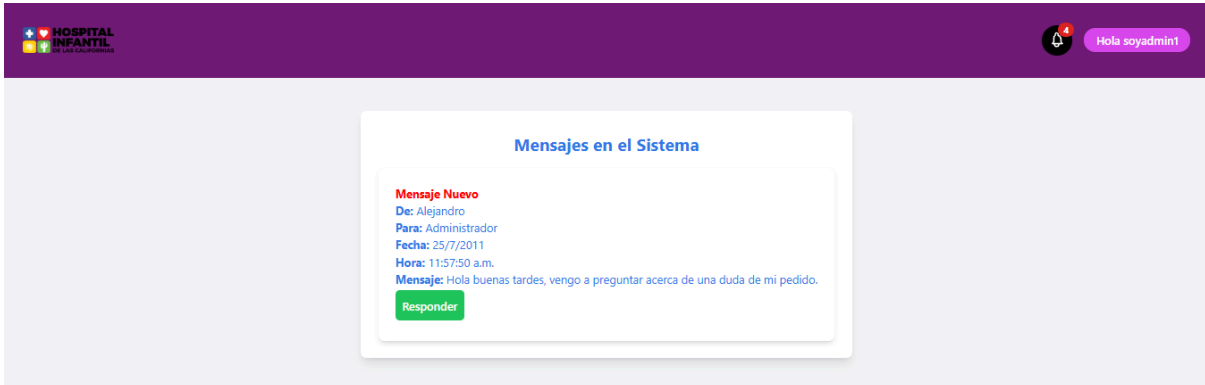
Envío de mensaje.

```
const handleSendMsg = async () => {
  if (msgContent === "") {
    alert("Debe escribir un mensaje primero");
    return;
  }
  try {
    const response = await axios.post('http://localhost:3000/message/new_msg', {
      sender_id: currentUser.user_id,
      receiver_id: receiverID,
      msg_content: msgContent,
      respondingTo: respondingTo,
    });
    if (response.data.success) {
      alert("Mensaje Enviado");
    }
    setResponding(null);
    window.location.reload();
  } catch (error) {
    alert("El mensaje no pudo ser enviado");
    console.error(error);
  }
  setWriting(false);
};
```



Obtener los mensajes de lado del usuario y el administrador.

```
const getMessages = async () => {
  if (currentUser.role === "general") {
    try {
      const response = await axios.get("http://localhost:3000/message/");
      if (response.data.messages) {
        const sent = response.data.messages.filter(msg => msg.sender_id === currentUser.user_id);
        const received = response.data.messages.filter(msg => msg.receiver_id === currentUser.user_id);
        setSentMsgs(sent);
        setReceivedMsgs(received);
      }
    } catch (error) {
      alert("No se pudo obtener la información de los mensajes");
      console.log("Error al obtener mensajes", error);
    }
  } else if (currentUser.role === "hic_admin") {
    try {
      const response = await axios.get("http://localhost:3000/message/");
      const msgsWithNames = await Promise.all(response.data.messages.map(async (msg) => {
        const patientDetails = await getPatientName(msg.sender_id);
        return { ...msg, patient_name: patientDetails ? patientDetails.user.name_ : "Desconocido" });
      }));
      const received = msgsWithNames.filter(msg => msg.receiver_id === null);
      setReceivedMsgs(received);
      setSentMsgs(msgsWithNames);
    } catch (error) {
      alert("No se pudo obtener la información de los mensajes");
      console.log("Error al obtener mensajes ->", error);
    }
  }
};
```



Justificación:

El envío de notificaciones y mensajes permiten una mejor comunicación entre usuario-administrador y usuario-plataforma obteniendo los mensajes guardados en la mostrando el mensaje dependiendo del rol del usuario y las notificaciones se manejan a través de los distintos eventos en la plataforma y luego se suben de forma automática a la sección de notificaciones para mantener una muestra de todas las notificaciones que conciernen a un usuario distinto pero todas son generales para el administrador.

1.7.- Actividad: Pruebas unitarias y de integración y Pruebas de aceptación.

Inicio: 6 de Mayo del 2025. Fin: 30 de Mayo del 2025.

Probatorio:

Muestra de pruebas unitarias realizadas.

```
PASS  _tests_/usuarios.test.js (8.151 s)
Módulo: Gestión de Usuarios (10 pruebas unitarias)
✓ PU-1 - createUser: rol 'hic_admin' con email/password válidos → HTTP 200 y user object (233 ms)
✓ PU-2 - createUser: rol 'hic_admin' sin email ni password → HTTP 400 (56 ms)
✓ PU-3 - createUser: rol 'general' sin matrícula → HTTP 400 (32 ms)
✓ PU-4 - createUser: rol no válido → HTTP 400 (41 ms)
✓ PU-5 - loginUser: rol 'admin' con email/password correctos → HTTP 200 y user object (111 ms)
✓ PU-6 - loginUser: rol 'admin' con contraseña incorrecta → HTTP 401 (56 ms)
✓ PU-7 - loginUser: rol 'general' con matrícula correcta → HTTP 200 y user object (66 ms)
✓ PU-8 - loginUser: rol 'general' con matrícula incorrecta → HTTP 401 (52 ms)
✓ PU-9 - updateUser: modificar name_ de usuario existente → HTTP 200 y user actualizado (69 ms)
✓ PU-10 - getUser: solicitar user_id que no existe → HTTP 404 (33 ms)

Test Suites: 1 passed, 1 total
Tests:       10 passed, 10 total
Snapshots:   0 total
Time:        8.49 s, estimated 16 s
Ran all test suites.
```

Muestra de las pruebas de integración.

Update HomePage.jsx

✓ Verified

de2739f

✎ <>

Update and rename Notification.jsx to NotificationsPanel.jsx

✓ Verified

167291b

✎ <>

Update App.jsx

✓ Verified

2c0ba7d

✎ <>

notificaciones GUI

9626037

✎ <>

:p

e34de4f

✎ <>

Commits on May 14, 2025

2do

5c2a378

✎ <>

GUI HistorialPagos

98adefc

✎ <>

:p

80adbe6

✎ <>

Commits on May 13, 2025

Pruebas de aceptación.

Se adjuntan en el **anexo** las fotos de la Prueba de aceptación con nuestro encargado de UR y la constancia de la entrega del proyecto en el Hospital Infantil de las Californias.

Justificación:

Durante esta actividad las pruebas son esenciales para el funcionamiento de nuestra plataforma, por lo que se realizaron distintas pruebas de integración para mostrar el funcionamiento general y que no sea único de un equipo, tanto pruebas unitarias para el

funcionamiento de cada unidad de la plataforma y la prueba de aceptación para mostrar al cliente el cumplimiento de cada módulo de la plataforma.

1.8.- Actividad: Despliegue y capacitación de personal y documentación técnica de la aplicación.

Inicio: 20 de Mayo del 2025. Fin: 30 de Mayo del 2025.

Probatorio:

Manual para el usuario:

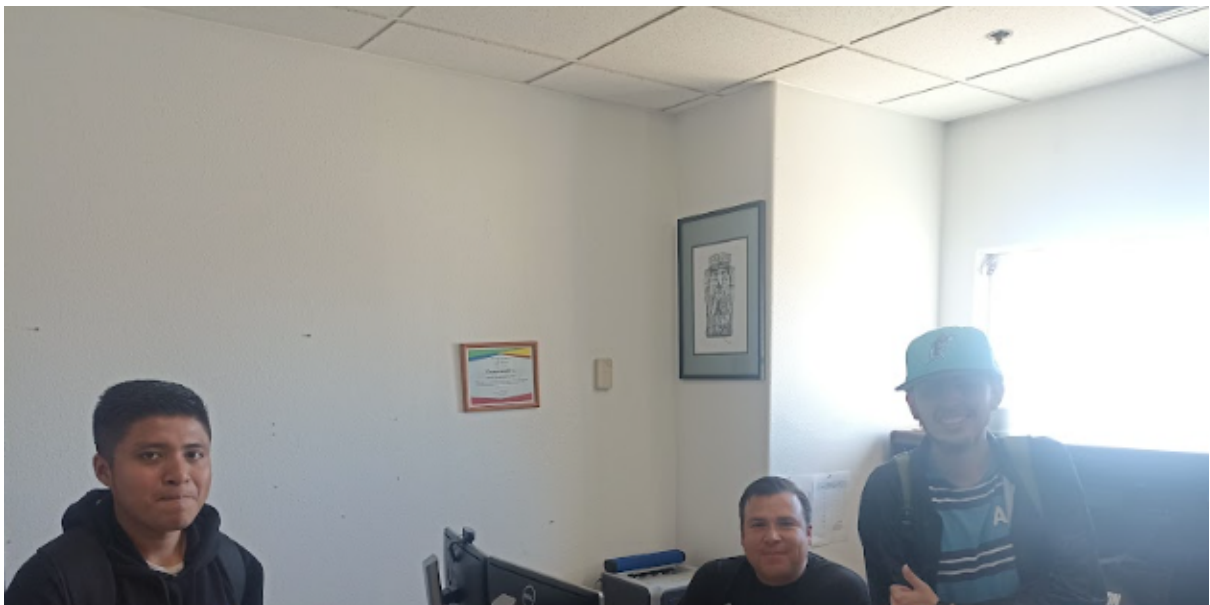
https://drive.google.com/file/d/1RclHnz-Z1WLnFx12qu7yiN5GyPo0mNHP/view?usp=drive_link

Justificación:

Durante el proceso de esta actividad nos quedamos con poco tiempo para el despliegue y capacitación del personal por lo que no se pudo realizar al completo esta actividad pero se pudo generar el manual de usuario para el uso del hospital para que puedan aprender a manejar y a resolver dudas del manejo de cada módulo de la plataforma.

2.- Evidencias fotográficas(anexo)

1ra evidencia:



2da evidencia:



3ra evidencia:



4ta evidencia:



Conclusión

Guerra Cervantes Omar Eduardo

Este proyecto fue una prueba para aplicar los conocimientos adquiridos durante la carrera. En la parte educativa, reforzamos habilidades clave en Análisis y Diseño de Sistemas, Ingeniería de Software y otros cursos fundamentales. También desarrollamos competencias en programación, lo que enriqueció nuestro aprendizaje. Al trabajar con el cliente, enfrentamos retos profesionales, cumpliendo plazos y entregando módulos funcionales al 100%. La presión del proyecto nos ayudó a mejorar nuestra capacidad de gestión. Fortalecimos habilidades blandas como la comunicación y el trabajo en equipo, ajustando soluciones de forma ágil. La experiencia nos preparó mejor para futuros proyectos en entornos profesionales. Además, brindamos un servicio útil a la comunidad y logramos cumplir con la mayoría de los módulos.

Navarro Quintero Ricardo

Durante el desarrollo del proyecto aprendí a utilizar nuevas tecnologías para el desarrollo web, en las clases impartidas durante la carrera solo se da una introducción al uso de html, css, javascript y php; por eso en esta aplicación se desarrolla con react que es intuitiva por

el uso de bibliotecas y nodejs con apoyo de algunas extensiones para mantener el server configurado.

En general me quedo con una buena experiencia, el trato con nuestra supervisora del área de farmacia fue respetuoso y nos apoyaba a contactar a otros departamentos para corroborar el uso adecuado de logotipo, verificar colores, etc. La presentación final en el hospital me permitió observar el entorno de trabajo y mejorar mis habilidades blandas para la exposición.

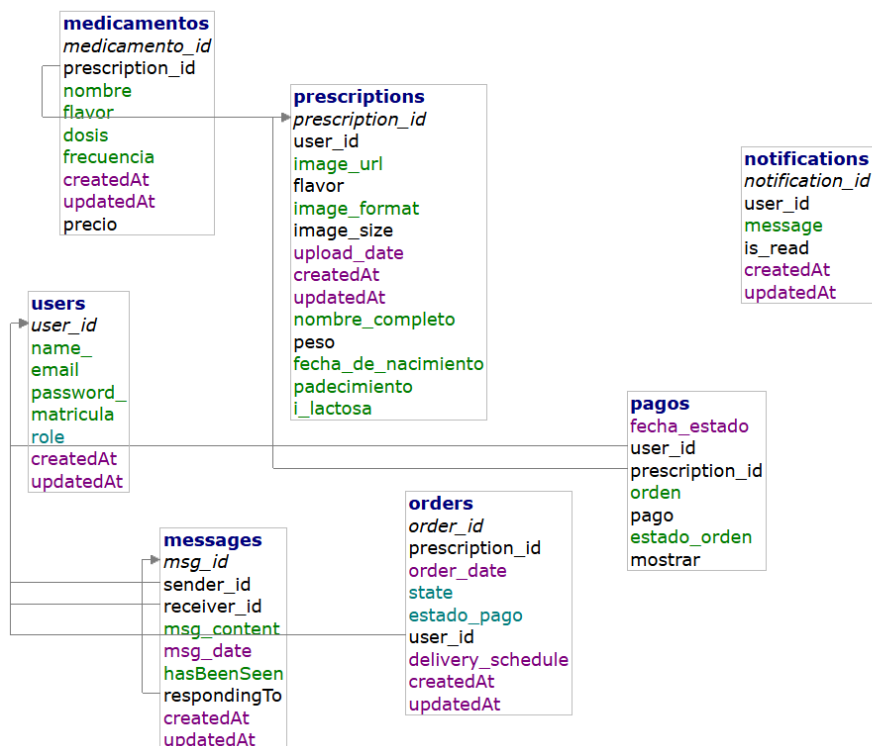
Tirzo Sotero Ricardo

En este proyecto se desarrolló una aplicación web utilizando React para el frontend y Node para el backend, así como javascript como lenguaje principal y Tailwind Css para estilo y responsividad. El desarrollo de este proyecto se logró implementando la metodología ágil “SCRUM” , esto permitió dividir el proyecto en módulos y a su vez en sprints. Esto permitió priorizar funciones y poder realizar cambios sin perder el objetivo del proyecto.

Gracias a esta metodología permitió negociar requisitos, adaptarse a los cambios imprevistos y priorizar las tareas de acorde al feedback del cliente. En conjunto, esta experiencia reforzó competencias técnicas y habilidades blandas.

Anexo

Diseño de la Base de Datos



Firma de aceptación del encargado de UR.

Competencia:

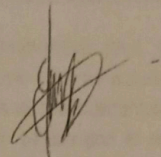
Diseñar e implementar un sistema software para automatizar la gestión de medicamentos y los procesos anexos, aplicando los principios de la ingeniería de software, haciendo análisis, diseño, implementación y pruebas, apegándose a estándares y guías de desarrollo, con actitud crítica, creatividad y responsabilidad.

Perfil del estudiante: Conocimientos de Programación en lenguajes de alto nivel, ingeniería de requerimientos, programación Web, manejo de bases de datos.

Plan de estudios: Ingeniero en Computación



R.F.C. FNC-920331-ET6
Ave. Alejandro von Humboldt # 11431
Cali: Barrio de Santa Juliana P.C.: C.P. 22430



Aprobado. QFB. Elvira Nieto Elcario

27 Mayo 2025

Constancia de proyecto en hospital infantil de las californias(1281859)



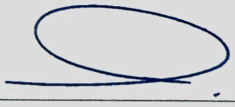
**HOSPITAL
INFANTIL
DE LAS CALIFORNIAS**

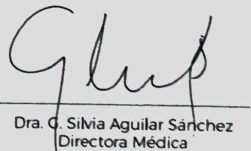
Otorga la presente

CONSTANCIA a:

Guerra Cervantes Omar Eduardo

Por este medio se hace constar que realizó el proyecto titulado "Proyecto Plataforma Web para Solicitud de Medicamentos Magistrales", para ser sometido por esta Unidad Receptora en la modalidad de Proyecto de Vinculación con Valor en Créditos en la Universidad Autónoma de Baja California periodo 2025-1.


Dr. Antonio Loaiza Martínez
Director General


Dra. C. Silvia Aguilar Sánchez
Directora Médica

Tijuana B.C., a 29 de mayo del 2025



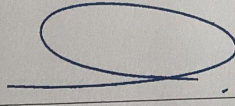
**HOSPITAL
INFANTIL
DE LAS CALIFORNIAS**

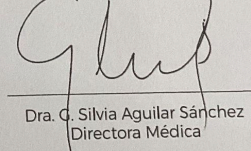
Otorga la presente

CONSTANCIA a:

Navarro Quintero Ricardo Antonio

Por este medio se hace constar que realizó el proyecto titulado "Proyecto Plataforma Web para Solicitud de Medicamentos Magistrales", para ser sometido por esta Unidad Receptora en la modalidad de Proyecto de Vinculación con Valor en Créditos en la Universidad Autónoma de Baja California periodo 2025-1.


Dr. Antonio Loaiza Martínez
Director General


Dra. C. Silvia Aguilar Sánchez
Directora Médica

Tijuana B.C., a 29 de mayo del 2025





Referencias

- ❖ *Metodologías de gestión de proyectos: 5 modelos provechosos.* (2023, February 14). Zendesk.
<https://www.zendesk.com.mx/blog/metodologias-gestion-proyectos/>
- ❖ *Node.js — Ejecuta JavaScript en cualquier parte.* (n.d.).
<https://nodejs.org/es>
- ❖ *React.* (n.d.). <https://es.react.dev/>
- ❖ *Vite.* (n.d.). Vitejs. <https://vite.dev/>
- ❖ Perú, P. (n.d.). *PROEFEX, comprometidos en aumentar el valor de tu empresa.* PROEFEX.
<https://proefexperu.com/blog/la-importancia-del-analisis-de-requerimientos-en-el-desarrollo-de-software>